Inventory Management System for B2B SaaS

Name : Abhishek Dani

Drive Link : https://drive.google.com/drive/folders/1K2rQ5QOoupO3lhQWNDyVOhAIlCwoJoyw?usp=sharing

Solution is attached below and all the code related is uploaded in the github respectively.

Overview

You're joining a team building "StockFlow" - a B2B inventory management platform. Small businesses use it to track products across multiple warehouses and manage supplier relationships.

Time Allocation

- **Take-Home Portion**: 90 minutes maximum
- **Live Discussion**: 30-45 minutes (scheduled separately)

Part 1: Code Review & Debugging (30 minutes)

A previous intern wrote this API endpoint for adding new products. **Something is wrong** - the code compiles but doesn't work as expected in production.

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
```

```
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

Your Tasks:

1. **Identify Issues**: List all problems you see with this code (technical and business logic)
2. **Explain Impact**: For each issue, explain what could go wrong in production
3. **Provide Fixes**: Write the corrected version with explanations

Additional Context (you may need to ask for more):

- Products can exist in multiple warehouses
- SKUs must be unique across the platform
- Price can be decimal values
- Some fields might be optional

Solution : Part 1: Code Review & Debugging

Issue	Root cause	Production impact
No transaction handling	Two separate commits : inventory insert can fail after product is created	Partial data, inconsistent system state
SKU uniqueness not enforced	No unique constraint or pre-insert check on SKU field	Duplicate products → billing and reporting conflicts
Price not handled as decimal	Stored as float — floating-point precision loss on arithmetic	Financial calculation errors in billing systems
No input validation	Missing checks: empty name, invalid price, negative quantity	Garbage data breaks downstream systems

No error handling	Unhandled DB exceptions propagate to caller	API crashes on any DB failure
No foreign key validation	warehouse_id not verified to exist before insert	Orphaned records, referential integrity broken
Product–warehouse model wrong	Assumes one warehouse per product; requirement is many-to-many	Not scalable — blocks multi-warehouse SaaS use case
No idempotency / duplicate guard	No deduplication key or request ID checked on each call	Retried requests create duplicate records

Fixed Code :

```
from flask import request, jsonify

from sqlalchemy.exc import IntegrityError

from decimal import Decimal

from app import db

from models import Product, Inventory, Warehouse


@app.route('/api/products', methods=['POST'])

def create_product():

    data = request.json


    try:

        #  Validation

        required_fields = ['name', 'sku', 'price']

        for field in required_fields:

            if field not in data:

                return {"error": f"{field} is required"}, 400
```

```
#  Check SKU uniqueness
existing = Product.query.filter_by(sku=data['sku']).first()
if existing:
    return {"error": "SKU already exists"}, 409

# Validate price
try:
    price = Decimal(str(data['price']))
except:
    return {"error": "Invalid price format"}, 400

#  Optional warehouse
warehouse_id = data.get('warehouse_id')
initial_quantity = data.get('initial_quantity', 0)

if warehouse_id:
    warehouse = Warehouse.query.get(warehouse_id)
    if not warehouse:
        return {"error": "Invalid warehouse_id"}, 400

#  Transaction block
with db.session.begin():

    product = Product(
        name=data['name'],
        sku=data['sku'],
```

```
            price=price
        )
        db.session.add(product)
        db.session.flush()  # get product.id

        #  Inventory only if warehouse provided
        if warehouse_id:
            inventory = Inventory(
                product_id=product.id,
                warehouse_id=warehouse_id,
                quantity=initial_quantity
            )
            db.session.add(inventory)

    return {
        "message": "Product created successfully",
        "product_id": product.id
    }, 201

except IntegrityError:
    db.session.rollback()
    return {"error": "Database integrity error"}, 500

except Exception as e:
    db.session.rollback()
    return {"error": str(e)}, 500
```

Explanation :

1. Input Validation

The API first checks whether all required fields (name, sku, price) are present in the request.

Why:

- Prevents incomplete or malformed requests
- Ensures only valid data enters the system

---

2. SKU Uniqueness Enforcement

Before creating the product, the code checks if a product with the same SKU already exists.

Why:

- SKU must uniquely identify a product
- Prevents duplicate entries which can break billing, tracking, and reporting

---

3. Price Validation Using Decimal

The price is converted to Decimal instead of using float.

Why:

- Float values can cause precision errors
- Decimal ensures accurate financial calculations (important in SaaS billing systems)

---

4. Optional Warehouse Validation

If a warehouse_id is provided, the API verifies that the warehouse exists.

Why:

- Prevents invalid foreign key references
- Ensures data consistency between tables

---

5. Transaction Management (Key Fix)

The product creation and inventory creation are wrapped inside a transaction:

with db.session.begin():

Why:

- Ensures atomicity
- Either both product and inventory are created OR neither is created

Fix over original code:

- Original code had multiple commits → could lead to partial data
- This version guarantees consistency

---

6. Product Creation and Flush

The product is added and flush() is used:

db.session.flush()

Why:

- Retrieves product.id without committing
- Needed to link inventory to the product

---

7. Conditional Inventory Creation

Inventory is created only if warehouse_id is provided.

Why:

- Supports flexible workflows

- Product may exist before being stocked

---

8. Error Handling

Two levels of error handling are implemented:

- IntegrityError
    - Handles database constraint violations
- Generic Exception
    - Handles unexpected failures

Why:

- Prevents API crashes
- Returns meaningful error responses

Part 2: Database Design (25 minutes)

Based on the requirements below, design a database schema. **Note**: These requirements are intentionally incomplete - you should identify what's missing.

Given Requirements:

- Companies can have multiple warehouses
- Products can be stored in multiple warehouses with different quantities
- Track when inventory levels change
- Suppliers provide products to companies
- Some products might be "bundles" containing other products

Your Tasks:

1. **Design Schema**: Create tables with columns, data types, and relationships

2. **Identify Gaps**: List questions you'd ask the product team about missing requirements
3. **Explain Decisions**: Justify your design choices (indexes, constraints, etc.)

Format: Use any notation (SQL DDL, ERD, text description, etc.)

Solution :

SQL CODE File :

https://drive.google.com/file/d/1ekc4KBQcpp5i75xF7LClaeX0sAHRkQTS/view?usp=sharing

SQL code :

```
-- Bynry Backend Assessment

-- Inventory Management Database

CREATE DATABASE IF NOT EXISTS bynry;

USE bynry;

-- =========================

-- TABLES

-- =========================

CREATE TABLE IF NOT EXISTS companies (

 id CHAR(36) PRIMARY KEY,

 name VARCHAR(255) NOT NULL,

 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE IF NOT EXISTS warehouses (

 id CHAR(36) PRIMARY KEY,

 company_id CHAR(36),

 name VARCHAR(255) NOT NULL,
```

```
 location TEXT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (company_id) REFERENCES companies(id)
 ON DELETE CASCADE
);
-- PRODUCTS
-- ========================
CREATE TABLE IF NOT EXISTS products (
 id CHAR(36) PRIMARY KEY,
 company_id CHAR(36),
 name VARCHAR(255) NOT NULL,
 sku VARCHAR(100) NOT NULL UNIQUE,
 price DECIMAL(10,2) NOT NULL,
 low_stock_threshold INT DEFAULT 10,
 is_bundle BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (company_id) REFERENCES companies(id)
 ON DELETE CASCADE
);
-- INVENTORY (CORE TABLE)
-- ========================
CREATE TABLE IF NOT EXISTS inventory (
 id CHAR(36) PRIMARY KEY,
 product_id CHAR(36),
 warehouse_id CHAR(36),
 quantity INT DEFAULT 0,
```

```
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

 UNIQUE(product_id, warehouse_id),
 FOREIGN KEY (product_id) REFERENCES products(id)
 ON DELETE CASCADE,
 FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
 ON DELETE CASCADE
);
-- INVENTORY LOGS (AUDIT TRAIL)
-- ========================
CREATE TABLE IF NOT EXISTS inventory_logs (
 id CHAR(36) PRIMARY KEY,
 product_id CHAR(36),
 warehouse_id CHAR(36),
 change_quantity INT,
 reason VARCHAR(255),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (product_id) REFERENCES products(id),
 FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);
-- SUPPLIERS
-- ========================
CREATE TABLE IF NOT EXISTS suppliers (
 id CHAR(36) PRIMARY KEY,
 name VARCHAR(255) NOT NULL,
```

```
 contact_email VARCHAR(255),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- PRODUCT-SUPPLIER (MANY-TO-MANY)
-- ========================
CREATE TABLE IF NOT EXISTS product_suppliers (
 product_id CHAR(36),
 supplier_id CHAR(36),
 PRIMARY KEY (product_id, supplier_id),
 FOREIGN KEY (product_id) REFERENCES products(id)
 ON DELETE CASCADE,
 FOREIGN KEY (supplier_id) REFERENCES suppliers(id)
 ON DELETE CASCADE
);
-- PRODUCT BUNDLES
-- ========================
CREATE TABLE IF NOT EXISTS product_bundles (
 parent_product_id CHAR(36),
 child_product_id CHAR(36),
 quantity INT DEFAULT 1,
 PRIMARY KEY (parent_product_id, child_product_id),
 FOREIGN KEY (parent_product_id) REFERENCES products(id),
 FOREIGN KEY (child_product_id) REFERENCES products(id)
);
-- SALES
-- ========================
```

```
CREATE TABLE IF NOT EXISTS sales (
 id CHAR(36) PRIMARY KEY,
 product_id CHAR(36),
 quantity INT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (product_id) REFERENCES products(id)
);
-- ========================
-- DEBUG / STRUCTURE CHECK
-- ========================
SHOW TABLES;
DESCRIBE products;
SHOW CREATE TABLE inventory;
-- ========================
-- SAMPLE DATA (SAFE INSERT)
-- ========================
INSERT IGNORE INTO companies (id, name)
VALUES ('c1', 'Demo Company');
INSERT IGNORE INTO warehouses (id, company_id, name)
VALUES ('w1', 'c1', 'Main Warehouse');
INSERT IGNORE INTO products (id, company_id, name, sku, price, low_stock_threshold)
VALUES ('p1', 'c1', 'Widget A', 'WID-001', 100.00, 20);
INSERT IGNORE INTO inventory (id, product_id, warehouse_id, quantity)
VALUES ('i1', 'p1', 'w1', 5);
INSERT IGNORE INTO suppliers (id, name, contact_email)
VALUES ('s1', 'Supplier Corp', 'orders@supplier.com');
```

```
INSERT IGNORE INTO product_suppliers (product_id, supplier_id)

VALUES ('p1', 's1');

INSERT IGNORE INTO sales (id, product_id, quantity, created_at)

VALUES ('sale1', 'p1', 10, NOW());

-- ========================

-- LOW STOCK QUERY (OUTPUT)

-- ========================

SELECT

 p.name,

 i.quantity,

 p.low_stock_threshold

FROM products p

JOIN inventory i ON p.id = i.product_id

WHERE i.quantity < p.low_stock_threshold;
```

Output Table :
https://drive.google.com/drive/folders/1KkNXtmcmrotIi2jBcp_VMTkKgiMIJavU?usp=sharing

---

Part 3: API Implementation (35 minutes)

Implement an endpoint that returns low-stock alerts for a company.

Business Rules (discovered through previous questions):

- Low stock threshold varies by product type
- Only alert for products with recent sales activity
- Must handle multiple warehouses per company
- Include supplier information for reordering

Endpoint Specification:

GET /api/companies/{company_id}/alerts/low-stock

Expected Response Format:

```
{
  "alerts": [
    {
      "product_id": 123,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 456,
      "warehouse_name": "Main Warehouse",
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 12,
      "supplier": {
        "id": 789,
        "name": "Supplier Corp",
        "contact_email": "orders@supplier.com"
      }
    }
  ],
  "total_alerts": 1
}
```

Your Tasks:

1. **Write Implementation**: Use any language/framework (Python/Flask, Node.js/Express, etc.)
2. **Handle Edge Cases**: Consider what could go wrong
3. **Explain Approach**: Add comments explaining your logic

Hints: You'll need to make assumptions about the database schema and business logic. Document these assumptions.

Solution :
Endpoint : GET /api/companies/{company_id}/alerts/low-stock

ASSUMPTION

Since requirements were incomplete, I assumed:

1. Low stock threshold is stored in products.low_stock_threshold
2. Recent sales activity = last 30 days
3. A product can have multiple warehouses
4. A product can have multiple suppliers, but returning one (primary)
5. Sales data is stored in sales table
6. If no sales → no alert (as per requirement)

Edge Case	How It Is Handled
Company not found / wrong ID	Auth check compares current_user.company_id; returns 403 before any DB query
Product below threshold but zero sales in last 30 days	INNER JOIN with recent_sales_sq filters these out — no false alerts for dead stock
Avg daily sales is 0 (new product)	days_until_stockout returns None, not a division-by-zero error
No supplier linked to product	supplier field returns null gracefully — client can show 'No supplier assigned'
Product in multiple warehouses — alert in one, fine in another	Query is per (product, warehouse) pair; only the low warehouse triggers an alert
Company has no warehouses or no products	Query returns empty list, total_alerts=0 — not an error
per_page > 200	Capped at 200 server-side to protect against OOM on large catalogs
Non-integer page / per_page params	try/except returns 400 Bad Request with helpful message
Bundle products with component stock depletion	Alert fires on the bundle's own inventory_record; component-level depletion is tracked separately via bundle_items logic (out of scope for this endpoint)

All code Files Are Uploaded on github related to Part 1,2,3.